JavaScript code initiates the request by calling the `XMLHttpRequest`'s `send` method ❶. The browser intercepts the request and initiates a preflight request ❷. If the server provides a valid preflight response ❸, the browser follows up by sending the actual request ❹. The server responds to the actual request ❺. This actual request is then sent to the calling JavaScript code for further processing ❻.

Let's wrap up our discussion on preflights by looking at a few things to keep in mind about preflight requests.

SUCCESSFUL PREFLIGHT != SUCCESSFUL REQUEST

The preflight response doesn't provide any insight into the success or failure of the actual request. A preflight could be successful, but the request could still fail for many reasons, such as a file not found, an authorization error, or a server issue. The preflight only ensures that the browser can make a cross-origin request to the server, and nothing more. The server is still free to reject the request for other reasons.

Think of an HTTP request as a set of Russian nesting dolls. Each doll contains a set of request headers that define the request behavior. Figure 4.19 shows how an HTTP request maps to this Russian nesting doll analogy.

The outermost doll is the CORS doll, and it contains the Origin header. The inner dolls can contain a variety of information, such as a cookie validating the user (cookie support in CORS will be covered in the next chapter). Even if the outermost CORS layer succeeds, the request may still fail while processing through one of the inner layers (for example, the cookie may have expired).

JAVASCRIPT CODE AND PREFLIGHTS

The preflight takes place solely between the browser and the server. There is no way for the JavaScript code to intercept the preflight response or get updates on its status. From the client code's perspective, the preflight is invisible. A failing preflight is akin to the actual request failing, even though the actual request is never made.

This is a useful feature, because it hides the complexity of preflight requests from the developer writing the client code. Figure 4.20 shows how both the client and the server view a CORS request with a preflight. There is complexity in answering a preflight

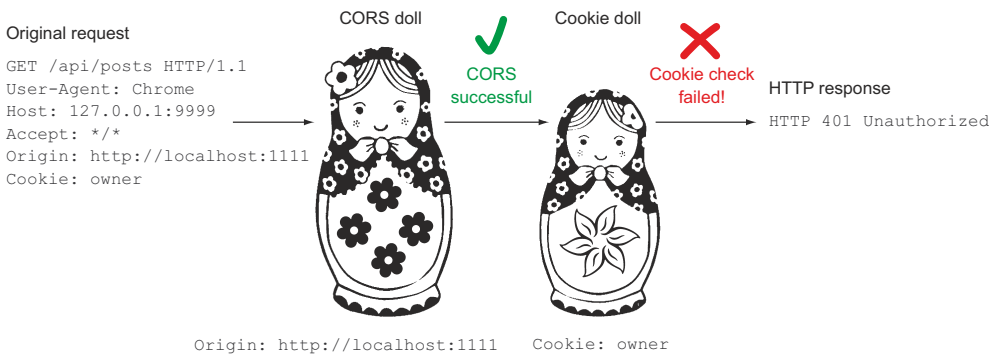


Figure 4.19 An HTTP request as a set of Russian dolls. Requests may fail for various reasons, and a successful preflight doesn't imply a successful request.